

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

**Системы ввода-вывода
Лабораторная работа №1**

Выполнили:

Вильданов Ильнур Наилевич
Бондарев Алексей Михайлович
Лобов Максим Павлович

Группа:

P3312

Преподаватель:

Быковский Сергей Вячеславович

Санкт-Петербург

2026 г.

Содержание

1. Задание	3
2. Введение	3
2.1. Цель работы	3
2.2. Задачи работы	3
3. Выполнение	3
3.1. Общая структура программы	3
3.2. Описание функций	3
3.2.1. Вызов OpenSBI: sbi_call	3
3.2.2. putchar	4
3.2.3. getchar	4
3.2.4. readline	4
3.2.5. puts	5
3.2.6. printf	5
3.3. Описание интерфейса вызова функций OpenSBI	5
3.3.1. 1. Get SBI specification version	5
3.3.2. 2. Get number of counters	6
3.3.3. 3. Get details of a counter	6
3.3.4. 4. System shutdown	6
3.4. Меню программы	7
4. Демонстрация	7
5. Вывод	9

1. Задание

2. Введение

2.1. Цель работы

Познакомиться с принципами организации ввода/вывода без операционной системы на примере компьютерной системы на базе процессора с архитектурой RISC-V и интерфейсом OpenSBI с использованием эмулятора QEMU.

2.2. Задачи работы

1. Реализовать функцию `putchar` для вывода данных в консоль.
2. Реализовать функцию `getchar` для получения данных из консоли.
3. На базе функций `putchar` и `getchar` реализовать меню для вызова функций OpenSBI по варианту 1.
4. Выполнить вызов всех пунктов меню и получить результаты их работы.
5. Оформить отчет по лабораторной работе.

3. Выполнение

3.1. Общая структура программы

Программа представляет собой bare-metal приложение для `qemu-system-riscv32`. После запуска OpenSBI выполняет инициализацию и передает управление программе. Далее отображается текстовое меню с вызовом функций OpenSBI, предусмотренных вариантом 1.

Реализованные пункты меню:

1. Get SBI specification version
2. Get number of counters
3. Get details of a counter
4. System shutdown

3.2. Описание функций

3.2.1. Вызов OpenSBI: `sbi_call`

Функция `sbi_call` является общей оберткой над инструкцией `ecall`. Аргументы вызова помещаются в регистры `a0`-`a7`, после чего управление передается OpenSBI. Результат возвращается в структуре `sbiret`, содержащей код ошибки и значение.

```
struct sbiret
sbi_call(long arg0, long arg1, long arg2, long arg3,
         long arg4, long arg5, long fid, long eid)
{
    register long a0 __asm__ ("a0") = arg0;
    register long a1 __asm__ ("a1") = arg1;
    register long a6 __asm__ ("a6") = fid;
    register long a7 __asm__ ("a7") = eid;

    __asm__ __volatile__ ("ecall"
        : "=r"(a0), "=r"(a1)
        : "r"(a0), "r"(a1), "r"(a6), "r"(a7)
```

```

        : "memory");

    return (struct sbiret){ .error = a0, .value = a1 };
}

```

3.2.2. putchar

Функция putchar реализует посимвольный вывод в консоль через legacy-вызов Console Putchar.

```

void
putchar(char ch)
{
    sbi_call(ch, 0, 0, 0, 0, 0, 0, SBI_ECALL_0_1_PUTCHAR);
}

```

3.2.3. getchar

Функция getchar реализует чтение символа из консоли через legacy-вызов Console Getchar. Если символ отсутствует, OpenSBI возвращает -1, поэтому выполняется опрос до получения корректного значения.

```

int
getchar(void)
{
    struct sbiret ret;

    do {
        ret = sbi_call(0, 0, 0, 0, 0, 0, 0, SBI_ECALL_0_1_GETCHAR);
    } while (ret.error == SBI_ERR_FAILED);

    return (int) ret.error;
}

```

3.2.4. readline

Функция readline используется для ввода строки с консоли. Поддерживается ввод символов, завершение по Enter и удаление по Backspace.

```

void
readline(char *buf, int max_len)
{
    int i = 0;
    char c;

    while (i < max_len - 1) {
        c = getchar();

        if (c == 8 || c == 127) {
            if (i > 0) {
                i--;
                buf[i] = '\0';
                putchar(8);
                putchar(' ');
                putchar(8);
            }
            continue;
        }
    }
}

```

```

    }

    if (c == '\r' || c == '\n') {
        putchar('\n');
        break;
    }

    if (c >= 32 && c < 127) {
        putchar(c);
        buf[i++] = c;
    }
}

buf[i] = '\0';
}

```

3.2.5. puts

Функция puts реализует вывод строки как последовательности символов с помощью putchar.

```

void
puts(const char *s)
{
    while (*s) {
        putchar(*s++);
    }
}

```

3.2.6. printf

Функция printf реализует упрощенный форматированный вывод. В программе она используется для вывода строк, целых чисел и шестнадцатеричных значений.

```

void
printf(const char *fmt, ...)
{
    va_list vars;
    va_start(vars, fmt);
    // разбор формата и вывод через putchar/puts
    va_end(vars);
}

```

3.3. Описание интерфейса вызова функций OpenSBI

3.3.1. 1. Get SBI specification version

Для получения версии спецификации используется Base Extension:

- EID = 0x10
- FID = 0x00

Из возвращенного значения выделяются major- и minor-части версии.

```

void
get_sbi_version()
{
    struct sbiret ret = sbi_call(0, 0, 0, 0, 0, 0, SBI_EXT_VER, SBI_EXT_BASE);
}

```

```

    long major = ret.value >> 24;
    long minor = ret.value & 0xFFFFF;
    printf("\nВерсия SBI: %d.%03d\n", major, minor);
}

```

3.3.2. 2. Get number of counters

Для получения числа счетчиков используется PMU Extension:

- EID = 0x504D55
- FID = 0x00

```

void
get_num_counters()
{
    struct sbiret ret = sbi_call(0, 0, 0, 0, 0, 0, SBI_EXT_CTR_NUM, SBI_EXT_PMU);
    printf("\nЧисло счётчиков: %d\n", ret.value);
}

```

3.3.3. 3. Get details of a counter

Для получения информации о конкретном счетчике используется PMU Extension:

- EID = 0x504D55
- FID = 0x01

Пользователь вводит номер счетчика, после чего программа вызывает OpenSBI и разбирает возвращенное значение counter_info.

```

void
get_counter_details()
{
    printf("\nВведите номер счётчика: ");
    char input[32];
    readline(input, sizeof(input));
    long counter_num = atoi(input);

    struct sbiret ret = sbi_call(counter_num, 0, 0, 0, 0, 0,
                                SBI_EXT_CTR_DTLS, SBI_EXT_PMU);

    if (ret.error) {
        printf("Ошибка: %d\n", ret.error);
        return;
    }

    unsigned long counter_info = ret.value;
    int type = (counter_info >> (__riscv_xlen - 1)) & 0x1;
    int width = (counter_info >> 12) & 0x3F;
    int csr = counter_info & 0xFFF;
    // вывод type, csr и width + 1
}

```

3.3.4. 4. System shutdown

Для завершения работы системы используется System Reset Extension:

- EID = 0x53525354
- FID = 0x00

```

void
system_shutdown()
{
    printf("Завершение работы системы...\n");
    sbi_call(0, 0, 0, 0, 0, 0, SBI_EXT_SHUTDOWN, SBI_EXT_SRST);
    while (1);
}

```

3.4. Меню программы

Основной цикл программы выводит меню, считывает введенную опцию и вызывает соответствующую функцию.

```

void
menu()
{
    while (1) {
        printf("\nМеню OpenSBI:\n");
        printf(" * 1. Get SBI specification version\n");
        printf(" * 2. Get number of counters\n");
        printf(" * 3. Get details of a counter\n");
        printf(" * 4. System shutdown\n");
        printf("Введите опцию: ");

        char input[32];
        readline(input, sizeof(input));
        int choice = atoi(input);

        switch (choice) {
            case 1: get_sbi_version(); break;
            case 2: get_num_counters(); break;
            case 3: get_counter_details(); break;
            case 4: system_shutdown(); break;
            default:
                printf("\nТакой опции нет. Попробуйте снова.\n");
        }

        printf("\n-----");
    }
}

```

4. Демонстрация

В данном разделе приводятся скриншоты консольного вывода при вызове каждого пункта меню.

```

OpenSBI v1.7
OpenSBI

Platform Name      : riscv-virtio,qemu
Platform Features  : mdecleg
Platform HART Count : 1
Platform FPI Device : aclicnt-mem
Platform Viter Device : aclicnt-mem @ 100000000
Platform Console Device : uart5200
Platform FPI Device : ---
Platform MMIO Device : ---
Platform Reboot Device : syscon-reboot
Platform Shutdown Device : syscon-powerrst
Platform Suspend Device : ---
Platform OPIC Device : ---
Firmware Base      : 0x00000000
Firmware Size       : 512 KB
Firmware RW Offset  : 0x40000
Firmware RW Size    : 57 KB
Firmware Heap Offset : 0x40000
Firmware Heap Size   : 37 KB (total), 2 KB (reserved), 10 KB (used), 24 KB (free)
Firmware Section Size : 4096 B (total), 1536 B (used), 2752 B (free)
Running SBI Version : 3.0
Standard SBI Extensions : time_rtno, ipi_base, hsm, srst, pmu, dcm, twt, legacy, dbtr, sse
Experimental SBI Extensions : none

Domain Name        : root
Domain Boot HART    : 0
Domain HARTs        : 0x
Domain Region0      : 0x00000000-0x00000000 M: (I,R,W) S/U: (R,W)
Domain Region1      : 0x10000000-0x10000000 M: (I,R,W) S/U: (R,W)
Domain Region2      : 0x20000000-0x20000000 M: (I,R,W) S/U: (I)
Domain Region3      : 0x30000000-0x30000000 M: (I,R,W) S/U: (I)
Domain Region4      : 0x40000000-0x40000000 M: (R,X) S/U: (I)
Domain Region5      : 0x50000000-0x50000000 M: (I,R,W) S/U: (R,W)
Domain Region6      : 0x60000000-0x60000000 M: (I,R,W) S/U: (R,W)
Domain Region7      : 0x70000000-0x70000000 M: (I) S/U: (R,W,X)
Domain Next Address  : 0x80200000
Domain Next Arg1     : 0x70000000
Domain Next Mode      : S-mode
Domain SysReset       : yes
Domain SysSuspend     : yes

Boot HART ID        : 0
Boot HART Domain     : root
Boot HART PMP Version : v1.2
Boot HART PMP ISA     : rv32imafdc
Boot HART PMP Extensions : satc, zicntr, zicpmp, zicboz, zicboz, edtrig, svedu
Boot HART PMP Count   : 16
Boot HART PMP Granularity : 2 bits
Boot HART PMP Address Bits : 32
Boot HART PMP Info     : 16 (0x00000000)
Boot HART Debug Triggers : 2 triggers
Boot HART MIDELEG     : 0x00000000
Boot HART MEDELEG     : 0x00000000

Menu OpenSBI:
* 1. Get SBI specification version
* 2. Get number of counters
* 3. Get details of a counter
* 4. System shutdown

```

Рис. 1. Запуск программы в QEMU и отображение главного меню

```

Меню OpenSBI:
* 1. Get SBI specification version
* 2. Get number of counters
* 3. Get details of a counter
* 4. System shutdown
[Введите опцию: 1

Версия SBI: 3.000

```

Рис. 2. Результат вызова пункта Get SBI specification version

```

Меню OpenSBI:
* 1. Get SBI specification version
* 2. Get number of counters
* 3. Get details of a counter
* 4. System shutdown
[Введите опцию: 2

Число счётчиков: 35

```

Рис. 3. Результат вызова пункта Get number of counters


```
Меню OpenSBI:
* 1. Get SBI specification version
* 2. Get number of counters
* 3. Get details of a counter
* 4. System shutdown
[Введите опцию: 3

[Введите номер счётчика: 7

Счётчик: 7
Детали:
* Тип: Аппаратура
* CSR: 3079
* Ширина: 64 бит
```

Рис. 4. Результат вызова пункта Get details of a counter для корректного номера счетчика

```
-----
Меню OpenSBI:
* 1. Get SBI specification version
* 2. Get number of counters
* 3. Get details of a counter
* 4. System shutdown
[Введите опцию: 4
Завершение работы системы...
gvdvddaa@MacBook-Pro-G io-lab1 % █
```

Рис. 5. Результат вызова пункта System shutdown

5. Вывод

В ходе выполнения лабораторной работы были реализованы функции `putchar` и `getchar`, обеспечивающие ввод и вывод данных через OpenSBI без использования операционной системы. На их основе была разработана программа с текстовым меню для вызова функций OpenSBI, предусмотренных вариантом 1. В результате работы были успешно получены версия спецификации SBI, число доступных счетчиков, сведения о выбранном счетчике, а также выполнено корректное завершение работы системы.